Lab 12 · Function Overriding & Abstract Classes

LAB 12

Function Overriding & Abstract Classes

Learning Objectives

By the end of this lab, you should be able to:

- Explain what function overriding is and how it differs from overloading.
- Use the virtual keyword to enable runtime polymorphism.
- Use a base-class pointer to call a derived class's overridden function.
- Use the override keyword for safety when redefining a virtual function.
- Define an abstract class using a pure virtual function (= 0).
- Understand why an abstract class cannot be instantiated directly.
- Force derived classes to implement specific functions through pure virtuals.

1. What Is Function Overriding?

Function overriding happens when a derived class provides its OWN version of a function that already exists in the base class — same name, same parameter list, same return type. When you call that function through a base-class pointer or reference, C++ runs the derived class's version, not the base's.

This is the essence of runtime polymorphism: the same line of code (emp->salary()) can call DIFFERENT functions depending on what kind of object emp actually points to at the moment.

Overriding vs Overloading — they sound similar, they aren't

Aspect	Overloading	Overriding
Same name?	Yes	Yes
Different parameters?	YES — that's the whole point	NO — must match exactly
Where it happens	Inside one class (or globally)	Across a base class and a derived class
When resolved	At compile time	At RUNTIME (with virtual)

The two keywords that make it work

- *virtual* — written in the BASE class function. It tells C++: "if a derived class redefines this, use the derived version even through a base pointer."

- *override* — written in the DERIVED class function (optional but recommended). The compiler will check that you really are overriding something — if you typo the name or change the parameter list, it'll catch the bug.

> 📘 **Without virtual, overriding doesn't really work**
> If the base function is NOT virtual, then *emp->salary()* will always call the BASE version — even when emp points to a Manager. This is called "static binding" and it defeats the whole purpose of polymorphism. **Always mark the base function virtual.**

2. Code 1 — Overriding the salary() Function

Goal: a base Employee class with a virtual salary function. Two derived classes — Manager and Developer — override it with their own values. We then use a SINGLE Employee pointer to call both.

The Code

```
#include <iostream>
using namespace std;

class Employee {
public:
    virtual void salary() {
        cout << "Base employee salary." << endl;
    }
};

class Manager : public Employee {
public:
    void salary() override {
        cout << "Manager salary: 50000 BDT" << endl;
    }
};

class Developer : public Employee {
public:
    void salary() override {
        cout << "Developer salary: 40000 BDT" << endl;
    }
};

int main() {
    Employee* emp;       // a base-class pointer
    Manager m;
    emp = &m;            // pointer now holds the address of a Manager
    emp->salary();       // calls Manager::salary — NOT Employee::salary

    Developer d;
    emp = &d;            // same pointer, now points to a Developer
    emp->salary();       // calls Developer::salary
}
```

Code Breakdown

Code	What it does
`virtual void salary()`	← The virtual keyword in the base class is what unlocks runtime polymorphism.
`void salary() override`	← Both derived classes redefine salary with the same signature. The override keyword asks the compiler to verify the match.

Code	What it does
`Employee* emp;`	← A pointer of the BASE type. It can point to any kind of Employee — including Manager or Developer.
`emp = &m;`	← Store the address of a Manager object inside an Employee pointer. Legal because Manager IS-A Employee.
`emp->salary();`	← The crucial line. Because salary is virtual, C++ checks AT RUNTIME what kind of object emp actually points to, then calls THAT class's version.

Expected Output

▶ Output

```
Manager salary: 50000 BDT
Developer salary: 40000 BDT
```

One pointer, many behaviors

Notice how the same variable *emp* produces different output depending on what it points to. This is the heart of polymorphism — code that works uniformly with many different types. If you had ten different employee types, this main function would still be just two pointer reassignments.

3. Code 2 — Overriding for Different Shapes

Goal: a Shape base class with virtual area and circumference functions. Three derived classes — Rectangle, Triangle, and Circle — each override both functions with their own formulas.

The Code

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() {
        cout << "Area for shape." << endl;
    }
    virtual void circumference() {
        cout << "Circumference for shape." << endl;
    }
};

class Rectangle : public Shape {
    float length, width;
public:
    Rectangle(float l, float w) { length = l; width = w; }
    void area() override {
        cout << "Rectangle Area: " << length * width << endl;
    }
    void circumference() override {
        cout << "Rectangle Circumference: " << 2 * (length + width) << endl;
    }
};

class Triangle : public Shape {
    float a, b, c, height;
public:
    Triangle(float x, float y, float z, float h) {
        a = x; b = y; c = z; height = h;
    }
    void area() override {
        cout << "Triangle Area: " << 0.5 * a * height << endl;
    }
    void circumference() override {
        cout << "Triangle Circumference: " << a + b + c << endl;
    }
};

class Circle : public Shape {
    float radius;
public:
    Circle(float r) { radius = r; }
    void area() override {
        cout << "Circle Area: " << 3.1416 * radius * radius << endl;
    }
```

```
    void circumference() override {
        cout << "Circle Circumference: " << 2 * 3.1416 * radius << endl;
    }
};

int main() {
    Rectangle rect(10, 5);
    Triangle  tri(6, 8, 10, 4);
    Circle    cir(7);

    cout << "Rectangle:" << endl;
    rect.area();
    rect.circumference();
    cout << "\nTriangle:" << endl;
    tri.area();
    tri.circumference();
    cout << "\nCircle:" << endl;
    cir.area();
    cir.circumference();
}
```

Code Breakdown

Code	What it does
`virtual void area() / circumference()`	← Two virtual functions in the base. Each derived class provides its own formula.
`Three derived classes`	← Rectangle, Triangle, Circle — each with its own private data and overriding functions.
`rect.area(), tri.area(), cir.area()`	← Called directly on the objects (no base pointer here). Each call runs its own class's version.
`The Shape base versions`	← The base "Area for shape." / "Circumference for shape." messages never actually print — every derived class overrides them. The base versions exist as a safe fallback.

Expected Output

▶ **Output**

```
Rectangle:
Rectangle Area: 50
Rectangle Circumference: 30

Triangle:
Triangle Area: 12
Triangle Circumference: 24

Circle:
Circle Area: 153.938
```

```
Circle Circumference: 43.9824
```

Notice: Shape's own area() is never useful

Look at Shape::area() — it just prints "Area for shape." That's not a real area; it's a placeholder. Every meaningful Shape MUST be a specific shape with real dimensions. This is exactly the situation that abstract classes are designed for — coming up next.

4. Abstract Classes — When the Base Has No Meaningful Implementation

Sometimes the base class can't sensibly do anything on its own. What is the area of "shape"? What is the salary of a generic "employee"? These are meaningless — only specific kinds of shapes and specific kinds of employees have real values.

C++ lets us express this with a pure virtual function — a virtual function with = 0; instead of a body. A class that contains even one pure virtual function becomes an abstract class.

Syntax of a pure virtual function

```
class Shape {
public:
    virtual double area() = 0;        // pure virtual – no body
    virtual void   display() = 0;     // pure virtual
};
```

What "abstract" means in practice

- **Cannot be instantiated.** *Shape s;* is a compile error. The class exists only to be inherited from.
- **Can be used as a pointer or reference type.** *Shape* s = new Circle(3);* is fine — the pointer is Shape, but the actual object is a Circle.
- **Forces derived classes to override.** Any class that inherits from Shape MUST provide its own area() and display() — otherwise THAT derived class is also abstract.

> **Why pure virtual is better than "empty body"**
> You could try writing *virtual void area() {}* with an empty body — but then derived classes are not FORCED to override it. They could silently inherit the do-nothing version. *= 0* is a contract: "every concrete derived class MUST provide its own version." The compiler enforces it.

5. Code 3 — An Abstract Shape Class

The Code

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() = 0;
    virtual void   display() = 0;
    ~Shape() {
        cout << "Destructor called" << endl;
    }
};

class Circle : public Shape {
```

```
    double r;
public:
    Circle(double r) { this->r = r; }
    double area() override { return 3.14159 * r * r; }
    void display() override {
        cout << "Circle area: " << area() << endl;
    }
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double w, double h) { this->w = w; this->h = h; }
    double area() override { return w * h; }
    void display() override {
        cout << "Rectangle area: " << area() << endl;
    }
};

int main() {
    // Shape s;          // ERROR — cannot instantiate an abstract class

    Shape* s = new Circle(3);
    s->display();
    s = new Rectangle(4, 5);
    s->display();
    delete s;

    Circle c1(4);
    c1.display();
    Rectangle r1(6, 7);
    r1.display();
}
```

Code Breakdown

Code	What it does
`virtual double area() = 0;`	← Pure virtual — makes Shape abstract. No body.
`~Shape()`	← Destructor. Will run automatically when an object goes out of scope (c1 and r1 at the end of main).
`Shape s;`	← If you uncomment this line, the program will not compile — abstract classes cannot be instantiated.
`Shape* s = new Circle(3);`	← PERFECTLY LEGAL — the pointer is Shape, the object is Circle. Polymorphism in action.
`s->display();`	← Calls Circle's display because display is virtual and s actually points to a Circle.

Code	What it does
`delete s;`	← Frees the heap-allocated Rectangle. (The earlier Circle was lost — see warning below.)
`c1.display(); r1.display();`	← Direct calls on stack-allocated derived objects — no base pointer needed here.

Expected Output

▶ **Output**

```
Circle area: 28.2743
Rectangle area: 20
Destructor called
Circle area: 50.2654
Rectangle area: 42
Destructor called
Destructor called
```

⚠ **Two important gotchas in this code**

1. **Memory leak:** the first *new Circle(3)* is overwritten by the new Rectangle without ever being deleted. In real code, always delete each heap object exactly once.

2. **Destructor should be virtual:** when you delete an object through a base pointer, only the BASE destructor runs unless the destructor is *virtual*. The rule of thumb: **if a class has any virtual function, give it a virtual destructor** — write *virtual ~Shape() { … }*.

6. Code 4 — Abstract Employee with Two Concrete Types

Goal: an Employee abstract class with two concrete implementations — FullTimeEmployee (monthly salary based on per-day rate) and PartTimeEmployee (hourly billing). This is the classic real-world use of abstract classes.

The Code

```
#include <iostream>
using namespace std;

class Employee {
protected:
    string name;
    int    id;
public:
    Employee(string name, int id) {
        this->name = name;
        this->id = id;
    }
    virtual double calculateSalary() = 0;    // pure virtual
```

```
    virtual void   display()         = 0;    // pure virtual
};

class FullTimeEmployee : public Employee {
    double PerDaySalary;
public:
    FullTimeEmployee(string name, int id, double salary)
        : Employee(name, id) {
        PerDaySalary = salary;
    }
    double calculateSalary() override {
        return PerDaySalary * 30;
    }
    void display() override {
        cout << "Full Time Employee" << endl;
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Salary: " << calculateSalary() << endl;
    }
};

class PartTimeEmployee : public Employee {
    int    hoursWorked;
    double hourlyRate;
public:
    PartTimeEmployee(string name, int id, int hours, double rate)
        : Employee(name, id) {
        hoursWorked = hours;
        hourlyRate  = rate;
    }
    double calculateSalary() override {
        return hoursWorked * hourlyRate;
    }
    void display() override {
        cout << "Part Time Employee" << endl;
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Hours Worked: " << hoursWorked << endl;
        cout << "Salary: " << calculateSalary() << endl;
    }
};

int main() {
    Employee* emp;
    emp = new FullTimeEmployee("Alice", 101, 50000);
    emp->display();
    cout << endl;

    emp = new PartTimeEmployee("Bob", 102, 25, 500);
    emp->display();
    cout << endl;

    FullTimeEmployee f1("Cat", 103, 65000);
    f1.display();
```

```
    cout << endl;

    PartTimeEmployee p1("Nabil", 104, 15, 400);
    p1.display();
}
```

Code Breakdown

Code	What it does
`protected: string name; int id;`	← protected (not private) so derived classes can access these fields directly in their display functions.
`Employee(string name, int id)`	← Constructor of the abstract base. Even though Employee can't be instantiated directly, its constructor still runs as part of every derived constructor.
`= 0;`	← Both calculateSalary and display are pure virtual — FullTimeEmployee and PartTimeEmployee MUST implement them.
`FullTimeEmployee(...) : Employee(name, id)`	← Constructor chaining to set name and id in the base portion.
`calculateSalary() override`	← Two completely different salary formulas, one per class — but they share the same interface so main() can treat them uniformly.
`Employee* emp = new FullTimeEmployee(...);`	← Polymorphic pointer — same pointer holds two completely different concrete types over the course of main().

Expected Output

▶ **Output**

```
Full Time Employee
Name: Alice
ID: 101
Salary: 1.5e+06

Part Time Employee
Name: Bob
ID: 102
Hours Worked: 25
Salary: 12500

Full Time Employee
Name: Cat
ID: 103
Salary: 1.95e+06
```

```
Part Time Employee
Name: Nabil
ID: 104
Hours Worked: 15
Salary: 6000
```

📘 Why this design is powerful

If you wanted to add a new employee type — say, *ContractEmployee* with a flat project fee — you just write the new class with its own *calculateSalary* and *display*. You don't touch FullTime, PartTime, or main() at all. The abstract Employee interface guarantees the new class plugs in seamlessly.

7. Quick Reference — virtual vs Pure Virtual

Aspect	virtual function	pure virtual function (= 0)
Has a body?	Yes	No (= 0 instead)
Can be called directly?	Yes	No
Class can be instantiated?	Yes	No — the class becomes abstract
Derived class MUST override?	No, override is optional	YES, otherwise derived class is also abstract
Typical use	Provide a default that derived classes may refine	Define an interface that derived classes MUST implement

8. Summary

Six things to take away from this lab:

- **Function overriding** lets a derived class redefine a function inherited from its base — same signature, different body.
- **virtual in the base class enables runtime polymorphism.** Without virtual, a base-class pointer always calls the base's version.
- **override in the derived class is optional but recommended** — it lets the compiler catch typos and signature mismatches.
- **A pure virtual function (= 0)** has no body. Any class containing one is abstract and cannot be instantiated directly.
- **Abstract classes define a contract** — derived classes MUST implement all pure virtual functions to become concrete.
- **If a class has virtual functions, give it a virtual destructor** so that *delete* through a base pointer cleans up properly.

9. Practice Tasks

Task 1 — Vehicle Hierarchy with Overriding

Create a base class Vehicle and two derived classes that override its functions.

Class Vehicle

- A virtual function start() that prints "Vehicle is starting..."
- A virtual function fuelType() that prints "Generic fuel."

Class Car (inherits Vehicle)

- Override start() to print "Car engine started with a key."
- Override fuelType() to print "Fuel type: Petrol".

Class ElectricBike (inherits Vehicle)

- Override start() to print "Bike powered on silently."
- Override fuelType() to print "Fuel type: Electric battery".

In main()

- Create a *Vehicle* v;* pointer.
- Point it first at a Car object, then at an ElectricBike object. Call start() and fuelType() through the pointer each time.
- Verify in the output that the derived versions run — not the base ones.

> **Hint for Task 1**
> Remember to mark the base functions *virtual*. Use *override* in both derived classes — try temporarily misspelling *start* as *Start* in one derived class and see how the compiler catches the mistake immediately.

Task 2 — Abstract Account Hierarchy

Design a banking system using an abstract Account base class.

Class Account (abstract)

- Protected data: accountNumber (int), holderName (string), balance (double). Constructor to initialize all three.
- **Pure virtual functions:** *calculateInterest()* (returns double) and *display()* (returns void).

Class SavingsAccount (inherits Account)

- Adds interestRate (double, e.g., 0.05 for 5%).
- calculateInterest() returns balance * interestRate.
- display() prints all account details including the calculated interest.

Class CurrentAccount (inherits Account)

- Adds overdraftLimit (double) but pays NO interest.
- calculateInterest() returns 0.
- display() prints all account details including the overdraft limit.

In main()

- Try writing *Account a;* — confirm the compiler refuses it, then comment it out.
- Use an *Account* acc;* pointer to point at one SavingsAccount and one CurrentAccount, calling display() on each.

- Also create one of each on the stack and call display() directly.

> 📘 **What you should observe**
>
> Each display() should print the correct details for its account type — SavingsAccount shows interest, CurrentAccount shows the overdraft limit. The same call through the same pointer type produces different output depending on the actual object. That is polymorphism doing its job.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST